

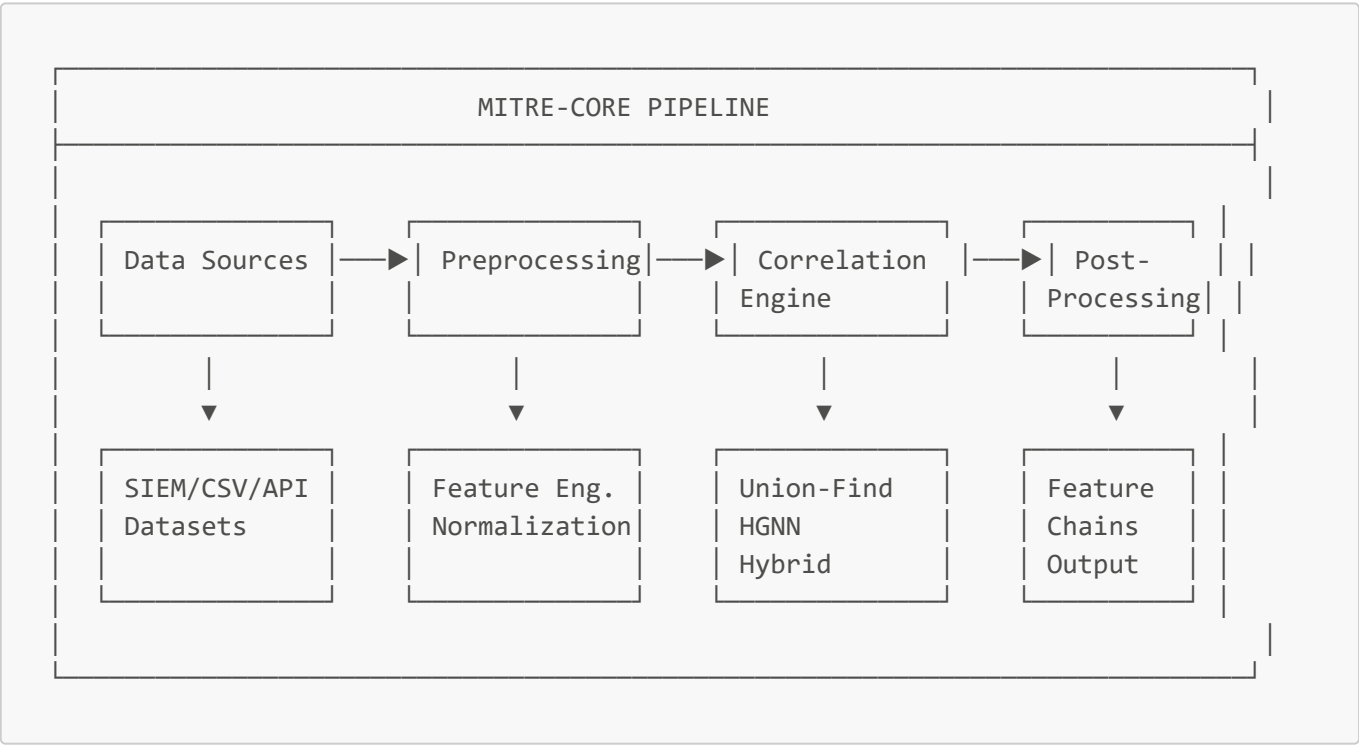
MITRE-CORE System Architecture Documentation

Table of Contents

- 1. [6-Stage Pipeline Overview](#)
- 2. [Stage 1: Data Ingestion](#)
- 3. [Stage 2: Preprocessing](#)
- 4. [Stage 3: Correlation Engine](#)
- 5. [Stage 4: Graph Construction](#)
- 6. [Stage 5: Post-Processing](#)
- 7. [Stage 6: Output Generation](#)
- 8. [Three Correlation Methods](#)
- 9. [Performance Comparison](#)
- 10. [Integration Examples](#)

6-Stage Pipeline Overview

MITRE-CORE processes security alerts through a systematic 6-stage pipeline that transforms raw SIEM events into actionable attack campaign clusters.



Stage 1: Data Ingestion

SIEM Connectors, CSV Uploads, API Endpoints

File: `siem/connectors.py`

MITRE-CORE supports multiple data ingestion methods through standardized connectors:

```
# Standard output schema for all connectors
STANDARD_COLUMNS = [
    "AlertId", "SourceAddress", "DestinationAddress", "DeviceAddress",
    "SourceUserName", "SourceHostName", "DeviceHostName", "DestinationHostName",
    "MalwareIntelAttackType", "AttackSeverity", "EndDate", "CustomerName",
]

class BaseSIEMConnector(abc.ABC):
    """Abstract base for every SIEM adapter."""

    name: str = "base"

    @abc.abstractmethod
    def connect(self, **kwargs) -> bool:
        """Establish connection to SIEM system."""
        pass

    @abc.abstractmethod
    def fetch_events(self, query: Dict[str, Any]) -> List[Dict[str, Any]]:
        """Retrieve events from SIEM."""
        pass

    @abc.abstractmethod
    def normalize_event(self, raw_event: Dict[str, Any]) -> Dict[str, Any]:
        """Convert raw SIEM event to standard schema."""
        pass
```

Supported Connectors:

1. Splunk Connector

```
class SplunkConnector(BaseSIEMConnector):
    name = "splunk"

    def connect(self, host: str, port: int, username: str, password: str) -> bool:
        """Connect to Splunk instance."""
        import splunklib.client as client
        import splunklib.results as results

        self.service = client.connect(
            host=host,
            port=port,
            username=username,
            password=password
        )
        return True

    def fetch_events(self, query: str, time_range: str = "-24h") -> List[Dict]:
        """Execute SPL query and return events."""
        search = self.service.jobs.create(query, earliest_time=time_range)
```

```

        while not search.is_done():
            time.sleep(0.5)

        reader = results.ResultsReader(search.results())
        events = [event for event in reader]
        return events

def normalize_event(self, raw_event: Dict) -> Dict:
    """Convert Splunk event to standard schema."""
    return {
        "AlertId": raw_event.get("_id", ""),
        "SourceAddress": raw_event.get("src_ip", ""),
        "DestinationAddress": raw_event.get("dest_ip", ""),
        "SourceUserName": raw_event.get("user", ""),
        "MalwareIntelAttackType": raw_event.get("signature", ""),
        "AttackSeverity": raw_event.get("severity", "medium"),
        "EndDate": raw_event.get("_time", ""),
    }

```

2. CSV Upload Handler

```

class CSVUploadHandler:
    """Handle bulk CSV file uploads."""

    def __init__(self, required_columns: List[str] = None):
        self.required_columns = required_columns or [
            "timestamp", "src_ip", "dst_ip", "username",
            "alert_type", "severity"
        ]

    def validate_csv(self, file_path: str) -> Tuple[bool, List[str]]:
        """Validate CSV format and required columns."""
        try:
            df = pd.read_csv(file_path)
            missing_cols = set(self.required_columns) - set(df.columns)

            if missing_cols:
                return False, f"Missing columns: {missing_cols}"

            return True, "CSV validation successful"
        except Exception as e:
            return False, f"CSV read error: {str(e)}"

    def process_csv(self, file_path: str) -> pd.DataFrame:
        """Process and normalize CSV to standard schema."""
        df = pd.read_csv(file_path)

        # Map CSV columns to standard schema
        column_mapping = {
            "timestamp": "EndDate",
            "src_ip": "SourceAddress",

```

```

        "dst_ip": "DestinationAddress",
        "username": "SourceUserName",
        "alert_type": "MalwareIntelAttackType",
        "severity": "AttackSeverity"
    }

    df = df.rename(columns=column_mapping)

    # Ensure all standard columns exist
    for col in STANDARD_COLUMNS:
        if col not in df.columns:
            df[col] = ""

    return df[STANDARD_COLUMNS]

```

3. REST API Endpoint

```

from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/ingest', methods=['POST'])
def ingest_alerts():
    """API endpoint for alert ingestion."""
    try:
        data = request.get_json()

        # Validate input format
        if not isinstance(data, list):
            return jsonify({"error": "Expected list of alerts"}), 400

        # Normalize each alert
        normalized_alerts = []
        for alert in data:
            normalized = normalize_api_alert(alert)
            normalized_alerts.append(normalized)

        # Convert to DataFrame for pipeline processing
        df = pd.DataFrame(normalized_alerts)

        # Trigger correlation pipeline
        from core.correlation_pipeline import CorrelationPipeline
        pipeline = CorrelationPipeline(method='auto')
        result = pipeline.correlate(df, ['SourceUserName'], ['SourceAddress'])

        return jsonify({
            "status": "success",
            "alerts_processed": len(normalized_alerts),
            "clusters_found": result.num_clusters,
            "correlation_method": result.method_used
        })

```

```

    except Exception as e:
        return jsonify({"error": str(e)}), 500

def normalize_api_alert(alert: Dict) -> Dict:
    """Normalize API alert to standard schema."""
    return {
        "AlertId": alert.get("id", str(uuid.uuid4())),
        "SourceAddress": alert.get("src_ip", ""),
        "DestinationAddress": alert.get("dst_ip", ""),
        "SourceUserName": alert.get("user", ""),
        "MalwareIntelAttackType": alert.get("type", ""),
        "AttackSeverity": alert.get("severity", "medium"),
        "EndDate": alert.get("timestamp", datetime.now().isoformat()),
    }

```

Stage 2: Preprocessing

Feature Engineering, Normalization, Temporal Extraction

File: `core/preprocessing.py`

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from datetime import datetime
import re

class AlertPreprocessor:
    """Comprehensive alert preprocessing pipeline."""

    def __init__(self):
        self.label_encoders = {}
        self.scalers = {}
        self.feature_columns = []

    def preprocess_data(self, df: pd.DataFrame) -> pd.DataFrame:
        """Main preprocessing pipeline."""
        print(f"Starting preprocessing with {len(df)} alerts")

        # Step 1: Data Cleaning
        df = self._clean_data(df)

        # Step 2: Feature Engineering
        df = self._extract_temporal_features(df)
        df = self._extract_network_features(df)
        df = self._encode_categorical_features(df)

        # Step 3: Normalization
        df = self._normalize_features(df)

```

```

# Step 4: Feature Selection
df = self._select_features(df)

print(f"Preprocessing complete. Final shape: {df.shape}")
return df

def _clean_data(self, df: pd.DataFrame) -> pd.DataFrame:
    """Data cleaning and missing value handling."""
    # Remove exact duplicates
    initial_count = len(df)
    df = df.drop_duplicates()
    print(f"Removed {initial_count - len(df)} duplicate alerts")

    # Handle missing values
    df = df.replace(["NIL", "null", "None", ""], np.nan)

    # Fill missing categorical values with 'UNKNOWN'
    categorical_cols = ['SourceUserName', 'SourceHostName',
'MalwareIntelAttackType']
    for col in categorical_cols:
        if col in df.columns:
            df[col] = df[col].fillna('UNKNOWN')

    # Fill missing IP addresses with '0.0.0.0'
    ip_cols = ['SourceAddress', 'DestinationAddress', 'DeviceAddress']
    for col in ip_cols:
        if col in df.columns:
            df[col] = df[col].fillna('0.0.0.0')

    return df

def _extract_temporal_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Extract temporal features from timestamps."""
    if 'EndDate' not in df.columns:
        return df

    # Convert timestamps to datetime
    df['timestamp'] = pd.to_datetime(df['EndDate'], errors='coerce')

    # Extract temporal features
    df['hour_of_day'] = df['timestamp'].dt.hour
    df['day_of_week'] = df['timestamp'].dt.dayofweek
    df['is_weekend'] = (df['timestamp'].dt.dayofweek >= 5).astype(int)
    df['is_business_hours'] = ((df['timestamp'].dt.hour >= 9) &
                                (df['timestamp'].dt.hour <= 17)).astype(int)

    # Cyclical encoding for temporal features
    df['hour_sin'] = np.sin(2 * np.pi * df['hour_of_day'] / 24)
    df['hour_cos'] = np.cos(2 * np.pi * df['hour_of_day'] / 24)
    df['day_sin'] = np.sin(2 * np.pi * df['day_of_week'] / 7)
    df['day_cos'] = np.cos(2 * np.pi * df['day_of_week'] / 7)

    return df

```

```

def _extract_network_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Extract network-based features."""
    # IP address categorization
    def categorize_ip(ip):
        if pd.isna(ip) or ip == '0.0.0.0':
            return 'unknown'

        # Private IP ranges
        private_ranges = [
            ('10.0.0.0', '10.255.255.255'),
            ('172.16.0.0', '172.31.255.255'),
            ('192.168.0.0', '192.168.255.255')
        ]

        for start, end in private_ranges:
            if self._ip_in_range(ip, start, end):
                return 'private'

        return 'public'

    df['src_ip_category'] = df['SourceAddress'].apply(categorize_ip)
    df['dst_ip_category'] = df['DestinationAddress'].apply(categorize_ip)

    # Username domain extraction
    def extract_email_domain(username):
        if pd.isna(username) or not isinstance(username, str):
            return 'unknown'

        email_regex = r"\S+@\S+\.\S+"
        if re.match(email_regex, username):
            return username[username.index('@') + 1:]
        return username

    df['username_domain'] = df['SourceUserName'].apply(extract_email_domain)

    return df

def _encode_categorical_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Encode categorical variables."""
    categorical_cols = [
        'MalwareIntelAttackType', 'AttackSeverity',
        'src_ip_category', 'dst_ip_category', 'username_domain'
    ]

    for col in categorical_cols:
        if col in df.columns:
            if col not in self.label_encoders:
                self.label_encoders[col] = LabelEncoder()

            # Handle unseen categories
            unique_values = df[col].unique()
            known_values = self.label_encoders[col].classes_

            # Add unknown category for new values

```

```

        if len(set(unique_values) - set(known_values)) > 0:
            all_values = list(known_values) + list(set(unique_values) -
set(known_values))
            self.label_encoders[col].fit(all_values)

        df[f'{col}_encoded'] = self.label_encoders[col].transform(df[col])

    return df

def _normalize_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Normalize numerical features."""
    numerical_cols = [
        'hour_sin', 'hour_cos', 'day_sin', 'day_cos',
        'MalwareIntelAttackType_encoded', 'AttackSeverity_encoded',
        'src_ip_category_encoded', 'dst_ip_category_encoded',
        'username_domain_encoded'
    ]

    # Filter existing columns
    numerical_cols = [col for col in numerical_cols if col in df.columns]

    if numerical_cols:
        if 'feature_scaler' not in self.scalers:
            self.scalers['feature_scaler'] = StandardScaler()

        df[numerical_cols] =
self.scalers['feature_scaler'].fit_transform(df[numerical_cols])
        self.feature_columns = numerical_cols

    return df

def _select_features(self, df: pd.DataFrame) -> pd.DataFrame:
    """Select and prepare final feature set."""
    # Ensure we have exactly 8 features as expected by HGNN
    if len(self.feature_columns) > 8:
        # Select top 8 features based on variance
        feature_variances = df[self.feature_columns].var()
        top_features = feature_variances.nlargest(8).index.tolist()
        self.feature_columns = top_features
    elif len(self.feature_columns) < 8:
        # Pad with zeros if fewer than 8 features
        missing_count = 8 - len(self.feature_columns)
        for i in range(missing_count):
            col_name = f'padding_feature_{i}'
            df[col_name] = 0.0
            self.feature_columns.append(col_name)

    return df

def _ip_in_range(self, ip: str, start_ip: str, end_ip: str) -> bool:
    """Check if IP is in given range."""
    try:
        ip_int = self._ip_to_int(ip)
        start_int = self._ip_to_int(start_ip)

```



```

        end_int = self._ip_to_int(end_ip)
        return start_int <= ip_int <= end_int
    except:
        return False

    def _ip_to_int(self, ip: str) -> int:
        """Convert IP string to integer."""
        try:
            return sum(int(octet) * (256 ** (3 - i)) for i, octet in
enumerate(ip.split('.')))
        except:
            return 0

    def get_feature_vector(self, alert_row: pd.Series) -> np.ndarray:
        """Extract 8-dimensional feature vector from alert."""
        features = []
        for col in self.feature_columns[:8]: # Ensure exactly 8 features
            features.append(alert_row.get(col, 0.0))

        return np.array(features, dtype=np.float32)

```

Stage 3: Correlation Engine

Union-Find / HGNN / Hybrid Methods

File: core/correlation_pipeline.py

```

from enum import Enum
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional, Literal
import pandas as pd
import numpy as np
import time

class CorrelationMethod(Enum):
    """Available correlation methods."""
    UNION_FIND = "union_find"
    HGNN = "hgnn"
    HYBRID = "hybrid"
    AUTO = "auto"

@dataclass
class CorrelationResult:
    """Result container for correlation operations."""
    data: pd.DataFrame
    method_used: str
    num_clusters: int
    runtime_seconds: float
    confidence_score: Optional[float] = None
    fallback_used: bool = False

```

```

class CorrelationPipeline:
    """
    Unified correlation pipeline supporting multiple methods.

    Features:
    - Automatic method selection based on data size and availability
    - Seamless fallback between methods
    - Consistent interface regardless of backend
    - Performance metrics and logging
    """

    def __init__(
        self,
        method: Literal["auto", "union_find", "hgnn", "hybrid"] = "auto",
        model_path: Optional[str] = None,
        device: Optional[str] = None,
        confidence_threshold: float = 0.5,
        hgnn_weight: float = 0.7,
        union_find_weight: float = 0.3,
        **kwargs
    ):
        """Initialize correlation pipeline."""
        self.method = method
        self.model_path = model_path
        self.device = device or ('cuda' if torch.cuda.is_available() else 'cpu')
        self.confidence_threshold = confidence_threshold
        self.hgnn_weight = hgnn_weight
        self.union_find_weight = union_find_weight

        # Initialize correlation engines
        self._initialize_engines()

    def _initialize_engines(self):
        """Initialize correlation engines based on configuration."""
        # Union-Find engine (always available)
        from core.correlation_indexer import enhanced_correlation
        self.union_find_engine = enhanced_correlation

        # HGNN engine (if available)
        self.hgnn_engine = None
        if self.model_path and Path(self.model_path).exists():
            try:
                from hgnn.hgnn_correlation import HGNNCorrelationEngine
                self.hgnn_engine = HGNNCorrelationEngine(
                    model_path=self.model_path,
                    device=self.device
                )
                print(f"Loaded HGNN model from {self.model_path}")
            except Exception as e:
                print(f"Failed to load HGNN model: {e}")

    def correlate(
        self,

```

```

        data: pd.DataFrame,
        usernames: List[str],
        addresses: List[str]
    ) -> CorrelationResult:
        """
        Run correlation analysis using selected method.

        Args:
            data: DataFrame with security alerts
            usernames: List of username column names
            addresses: List of IP address column names

        Returns:
            CorrelationResult with clustered data and metadata
        """
        start_time = time.time()

        # Determine method to use
        method_to_use = self._select_method(data)
        print(f"Using correlation method: {method_to_use}")

        # Run correlation
        if method_to_use == "union_find":
            result_data = self._run_union_find(data, usernames, addresses)
        elif method_to_use == "hgnn":
            result_data = self._run_hgnn(data, usernames, addresses)
        elif method_to_use == "hybrid":
            result_data = self._run_hybrid(data, usernames, addresses)
        else:
            raise ValueError(f"Unknown correlation method: {method_to_use}")

        # Calculate metrics
        runtime = time.time() - start_time
        num_clusters = result_data['cluster'].nunique()

        return CorrelationResult(
            data=result_data,
            method_used=method_to_use,
            num_clusters=num_clusters,
            runtime_seconds=runtime,
            confidence_score=self._calculate_confidence(result_data),
            fallback_used=(method_to_use != self.method and self.method != "auto")
        )

    def _select_method(self, data: pd.DataFrame) -> str:
        """Select correlation method based on data size and availability."""
        if self.method != "auto":
            return self.method.value

        num_alerts = len(data)

        # Auto-selection logic
        if num_alerts < 100:
            return "union_find"

```

```
        elif num_alerts > 1000 and self.hgnn_engine:
            return "hgnn"
        elif self.hgnn_engine:
            return "hybrid"
        else:
            return "union_find"

    def _run_union_find(
        self,
        data: pd.DataFrame,
        usernames: List[str],
        addresses: List[str]
    ) -> pd.DataFrame:
        """Run Union-Find correlation."""
        result_data = data.copy()

        # Use enhanced correlation function
        clustered_data = self.union_find_engine(result_data, usernames, addresses)

        return clustered_data

    def _run_hgnn(
        self,
        data: pd.DataFrame,
        usernames: List[str],
        addresses: List[str]
    ) -> pd.DataFrame:
        """Run HGNN correlation."""
        if not self.hgnn_engine:
            raise ValueError("HGNN model not loaded")

        # Preprocess data for HGNN
        preprocessor = AlertPreprocessor()
        processed_data = preprocessor.preprocess_data(data)

        # Run HGNN correlation
        result_data = self.hgnn_engine.correlate(
            processed_data,
            usernames,
            addresses,
            confidence_threshold=self.confidence_threshold
        )

        return result_data

    def _run_hybrid(
        self,
        data: pd.DataFrame,
        usernames: List[str],
        addresses: List[str]
    ) -> pd.DataFrame:
        """Run hybrid correlation (Union-Find + HGNN)."""
        # Run Union-Find first
        uf_result = self._run_union_find(data, usernames, addresses)
```

```

# Run HGNN
hgnn_result = self._run_hgnn(data, usernames, addresses)

# Combine results
combined_data = self._combine_results(
    uf_result, hgnn_result,
    self.hgnn_weight, self.union_find_weight
)

return combined_data

def _combine_results(
    self,
    uf_result: pd.DataFrame,
    hgnn_result: pd.DataFrame,
    hgnn_weight: float,
    uf_weight: float
) -> pd.DataFrame:
    """Combine Union-Find and HGNN results."""
    combined = uf_result.copy()

    # Create weighted cluster assignments
    uf_clusters = uf_result['cluster'].values
    hgnn_clusters = hgnn_result['cluster'].values

    # Simple weighted combination (could be made more sophisticated)
    final_clusters = []
    for i in range(len(uf_clusters)):
        if hgnn_weight > uf_weight:
            final_clusters.append(hgnn_clusters[i])
        else:
            final_clusters.append(uf_clusters[i])

    combined['cluster'] = final_clusters
    combined['method'] = 'hybrid'

    return combined

def _calculate_confidence(self, result_data: pd.DataFrame) -> float:
    """Calculate overall confidence score."""
    # Simple confidence based on cluster quality
    cluster_sizes = result_data['cluster'].value_counts()

    # Confidence based on average cluster size (larger clusters = higher
confidence)
    avg_cluster_size = cluster_sizes.mean()
    confidence = min(avg_cluster_size / 10.0, 1.0) # Normalize to [0,1]

    return confidence

```

Stage 4: Graph Construction

Heterogeneous Graph Building (HGNN only)

File: hgnn/hgnn_correlation.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.nn import HeteroConv, GATConv, Linear, global_mean_pool
from torch_geometric.data import HeteroData
from typing import Dict, List, Tuple, Optional
import pandas as pd
import numpy as np
from collections import defaultdict

class AlertToGraphConverter:
    """
    Convert security alerts to heterogeneous graph representation.

    Graph Schema:
    - Node Types: alert, user, host, ip
    - Edge Types: ownership, involvement, temporal proximity
    """

    def __init__(self, device: str = 'cpu'):
        self.device = device
        self.node_mappings = {}
        self.edge_mappings = {}

    def convert_to_graph(self, df: pd.DataFrame) -> HeteroData:
        """
        Convert DataFrame of alerts to heterogeneous graph.

        Args:
            df: Preprocessed alert DataFrame

        Returns:
            HeteroData object with graph structure
        """
        data = HeteroData()

        # Step 1: Extract nodes
        alert_nodes = self._extract_alert_nodes(df)
        user_nodes = self._extract_user_nodes(df)
        host_nodes = self._extract_host_nodes(df)
        ip_nodes = self._extract_ip_nodes(df)

        # Step 2: Create node features
        data['alert'].x = self._create_alert_features(df, alert_nodes)
        data['user'].x = self._create_user_features(user_nodes)
        data['host'].x = self._create_host_features(host_nodes)
```

```

        data['ip'].x = self._create_ip_features(ip_nodes)

        # Step 3: Create edges
        edge_index_dict = self._create_edges(df, alert_nodes, user_nodes,
host_nodes, ip_nodes)
        data.edge_index_dict = edge_index_dict

        # Step 4: Create edge attributes
        data.edge_attr_dict = self._create_edge_attributes(edge_index_dict)

        return data.to(self.device)

    def _extract_alert_nodes(self, df: pd.DataFrame) -> List[str]:
        """Extract alert node identifiers."""
        alert_ids = [f"alert_{i}" for i in range(len(df))]
        self.node_mappings['alert'] = {alert_id: i for i, alert_id in
enumerate(alert_ids)}
        return alert_ids

    def _extract_user_nodes(self, df: pd.DataFrame) -> List[str]:
        """Extract unique user nodes."""
        users = set()
        for username in ['SourceUserName', 'DestinationUserName']:
            if username in df.columns:
                users.update(df[username].dropna().unique())

        user_list = [f"user_{u}" for u in users if u != 'UNKNOWN']
        self.node_mappings['user'] = {user: i for i, user in enumerate(user_list)}
        return user_list

    def _extract_host_nodes(self, df: pd.DataFrame) -> List[str]:
        """Extract unique host nodes."""
        hosts = set()
        for hostname in ['SourceHostName', 'DestinationHostName',
'DeviceHostName']:
            if hostname in df.columns:
                hosts.update(df[hostname].dropna().unique())

        host_list = [f"host_{h}" for h in hosts if h != 'UNKNOWN']
        self.node_mappings['host'] = {host: i for i, host in enumerate(host_list)}
        return host_list

    def _extract_ip_nodes(self, df: pd.DataFrame) -> List[str]:
        """Extract unique IP nodes."""
        ips = set()
        for ip_col in ['SourceAddress', 'DestinationAddress', 'DeviceAddress']:
            if ip_col in df.columns:
                ips.update(df[ip_col].dropna().unique())

        ip_list = [f"ip_{ip}" for ip in ips if ip != '0.0.0.0']
        self.node_mappings['ip'] = {ip: i for i, ip in enumerate(ip_list)}
        return ip_list

    def _create_alert_features(self, df: pd.DataFrame, alert_nodes: List[str]) ->

```

```

torch.Tensor:
    """Create alert node features (8-dimensional)."""
    features = []

    for i, alert_id in enumerate(alert_nodes):
        alert_row = df.iloc[i]

        # Extract 8-dimensional feature vector
        feature_vector = [
            alert_row.get('hour_sin', 0.0),
            alert_row.get('hour_cos', 0.0),
            alert_row.get('day_sin', 0.0),
            alert_row.get('day_cos', 0.0),
            alert_row.get('MalwareIntelAttackType_encoded', 0.0),
            alert_row.get('AttackSeverity_encoded', 0.0),
            alert_row.get('src_ip_category_encoded', 0.0),
            alert_row.get('username_domain_encoded', 0.0)
        ]

        features.append(feature_vector)

    return torch.tensor(features, dtype=torch.float32)

def _create_user_features(self, user_nodes: List[str]) -> torch.Tensor:
    """Create user node features (32-dimensional)."""
    # Simple user features based on username characteristics
    features = []
    for user in user_nodes:
        # Extract user type from username
        username = user.replace('user_', '')

        feature_vector = [
            len(username) / 50.0, # Username length (normalized)
            1.0 if '@' in username else 0.0, # Is email
            1.0 if 'admin' in username.lower() else 0.0, # Is admin
            1.0 if 'service' in username.lower() else 0.0, # Is service
            account
        ] + [0.0] * 28 # Pad to 32 dimensions

        features.append(feature_vector)

    return torch.tensor(features, dtype=torch.float32)

def _create_host_features(self, host_nodes: List[str]) -> torch.Tensor:
    """Create host node features (32-dimensional)."""
    features = []
    for host in host_nodes:
        hostname = host.replace('host_', '')

        feature_vector = [
            len(hostname) / 50.0, # Hostname length (normalized)
            1.0 if '-' in hostname else 0.0, # Contains hyphen
            1.0 if hostname.isdigit() else 0.0, # Is numeric
            hash(hostname) % 1000 / 1000.0, # Hash-based feature

```



```

        ] + [0.0] * 28 # Pad to 32 dimensions

        features.append(feature_vector)

    return torch.tensor(features, dtype=torch.float32)

def _create_ip_features(self, ip_nodes: List[str]) -> torch.Tensor:
    """Create IP node features (32-dimensional)."""
    features = []
    for ip in ip_nodes:
        ip_addr = ip.replace('ip_', '')

        # Extract IP-based features
        octets = ip_addr.split('.')
        feature_vector = [
            int(octets[0]) / 255.0 if len(octets) > 0 else 0.0, # First octet
            int(octets[1]) / 255.0 if len(octets) > 1 else 0.0, # Second
            int(octets[2]) / 255.0 if len(octets) > 2 else 0.0, # Third octet
            int(octets[3]) / 255.0 if len(octets) > 3 else 0.0, # Fourth
            1.0 if self._is_private_ip(ip_addr) else 0.0, # Is private IP
        ] + [0.0] * 27 # Pad to 32 dimensions

        features.append(feature_vector)

    return torch.tensor(features, dtype=torch.float32)

def _create_edges(
    self,
    df: pd.DataFrame,
    alert_nodes: List[str],
    user_nodes: List[str],
    host_nodes: List[str],
    ip_nodes: List[str]
) -> Dict[Tuple[str, str, str], torch.Tensor]:
    """Create heterogeneous edges."""
    edge_index_dict = {}

    # Alert -> User edges
    alert_user_edges = []
    user_alert_edges = []

    # Alert -> Host edges
    alert_host_edges = []
    host_alert_edges = []

    # Alert -> IP edges
    alert_ip_edges = []
    ip_alert_edges = []

    # Alert -> Alert edges (temporal and IP sharing)
    alert_alert_edges = []

```

```

for i, alert_row in df.iterrows():
    alert_id = f"alert_{i}"

    # Alert -> User edges
    for user_col in ['SourceUserName', 'DestinationUserName']:
        if user_col in alert_row and pd.notna(alert_row[user_col]):
            user_id = f"user_{alert_row[user_col]}"
            if user_id in self.node_mappings['user']:
                alert_idx = self.node_mappings['alert'][alert_id]
                user_idx = self.node_mappings['user'][user_id]

                alert_user_edges.append([alert_idx, user_idx])
                user_alert_edges.append([user_idx, alert_idx])

    # Alert -> Host edges
    for host_col in ['SourceHostName', 'DestinationHostName',
'DeviceHostName']:
        if host_col in alert_row and pd.notna(alert_row[host_col]):
            host_id = f"host_{alert_row[host_col]}"
            if host_id in self.node_mappings['host']:
                alert_idx = self.node_mappings['alert'][alert_id]
                host_idx = self.node_mappings['host'][host_id]

                alert_host_edges.append([alert_idx, host_idx])
                host_alert_edges.append([host_idx, alert_idx])

    # Alert -> IP edges
    for ip_col in ['SourceAddress', 'DestinationAddress',
'DeviceAddress']:
        if ip_col in alert_row and pd.notna(alert_row[ip_col]):
            ip_id = f"ip_{alert_row[ip_col]}"
            if ip_id in self.node_mappings['ip']:
                alert_idx = self.node_mappings['alert'][alert_id]
                ip_idx = self.node_mappings['ip'][ip_id]

                alert_ip_edges.append([alert_idx, ip_idx])
                ip_alert_edges.append([ip_idx, alert_idx])

    # Alert -> Alert edges (temporal proximity)
    for i in range(len(df)):
        for j in range(i + 1, len(df)):
            alert_i = df.iloc[i]
            alert_j = df.iloc[j]

            # Check temporal proximity (within 1 hour)
            if 'timestamp' in alert_i and 'timestamp' in alert_j:
                time_diff = abs((alert_i['timestamp'] -
alert_j['timestamp']).total_seconds())
                if time_diff < 3600: # 1 hour
                    alert_alert_edges.append([i, j])
                    alert_alert_edges.append([j, i])

    # Check IP sharing
    shared_ips = set()

```

```

        for ip_col in ['SourceAddress', 'DestinationAddress']:
            if ip_col in alert_i and ip_col in alert_j:
                if alert_i[ip_col] == alert_j[ip_col] and
pd.notna(alert_i[ip_col]):
                    shared_ips.add(alert_i[ip_col])

        if shared_ips:
            alert_alert_edges.append([i, j])
            alert_alert_edges.append([j, i])

    # Convert to tensors
    if alert_user_edges:
        edge_index_dict[('alert', 'owned_by', 'user')] =
torch.tensor(alert_user_edges).T
        edge_index_dict[('user', 'owns', 'alert')] =
torch.tensor(user_alert_edges).T

    if alert_host_edges:
        edge_index_dict[('alert', 'generated_by', 'host')] =
torch.tensor(alert_host_edges).T
        edge_index_dict[('host', 'generates', 'alert')] =
torch.tensor(host_alert_edges).T

    if alert_ip_edges:
        edge_index_dict[('alert', 'involves', 'ip')] =
torch.tensor(alert_ip_edges).T
        edge_index_dict[('ip', 'involved_in', 'alert')] =
torch.tensor(ip_alert_edges).T

    if alert_alert_edges:
        edge_index_dict[('alert', 'temporal_near', 'alert')] =
torch.tensor(alert_alert_edges).T

    return edge_index_dict

def _create_edge_attributes(self, edge_index_dict: Dict) -> Dict:
    """Create edge attributes for each edge type."""
    edge_attr_dict = {}

    for edge_type, edge_index in edge_index_dict.items():
        # Simple edge attributes (can be extended)
        num_edges = edge_index.shape[1]

        # Edge weight based on edge type
        if 'temporal' in edge_type[1]:
            # Temporal edges have weights based on time proximity
            weights = torch.ones(num_edges) * 0.8
        elif 'shares_ip' in edge_type[1]:
            # IP sharing edges have high weight
            weights = torch.ones(num_edges) * 0.9
        else:
            # Ownership/involvement edges have medium weight
            weights = torch.ones(num_edges) * 0.7

```

```

        edge_attr_dict[edge_type] = weights

    return edge_attr_dict

def _is_private_ip(self, ip: str) -> bool:
    """Check if IP is in private range."""
    try:
        octets = [int(o) for o in ip.split('.')]
        return (
            (octets[0] == 10) or
            (octets[0] == 172 and 16 <= octets[1] <= 31) or
            (octets[0] == 192 and octets[1] == 168)
        )
    except:
        return False

class MITREHeteroGNN(nn.Module):
    """
    Heterogeneous Graph Neural Network for alert correlation.

    Architecture:
    - Alert Encoder (8 -> 64 dimensions)
    - Heterogeneous GAT Layers
    - Global Pooling
    - Cluster Classifier
    """

    def __init__(
        self,
        hidden_channels: int = 64,
        num_heads: int = 4,
        num_layers: int = 2,
        num_clusters: int = 10
    ):
        super().__init__()

        # Node encoders
        self.alert_encoder = Linear(8, hidden_channels)
        self.user_encoder = Linear(32, hidden_channels)
        self.host_encoder = Linear(32, hidden_channels)
        self.ip_encoder = Linear(32, hidden_channels)

        # Heterogeneous GAT layers
        self.convs = nn.ModuleList()
        for _ in range(num_layers):
            conv = HeteroConv({
                ('alert', 'owned_by', 'user'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
                ('user', 'owns', 'alert'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
                ('alert', 'generated_by', 'host'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
                ('host', 'generates', 'alert'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),

```

```

        ('alert', 'involves', 'ip'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
        ('ip', 'involved_in', 'alert'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
        ('alert', 'temporal_near', 'alert'): GATConv(hidden_channels,
hidden_channels // num_heads, heads=num_heads),
    }, aggr='mean')
    self.convs.append(conv)

# Cluster classifier
self.classifier = nn.Sequential(
    Linear(hidden_channels, hidden_channels // 2),
    nn.ReLU(),
    nn.Dropout(0.2),
    Linear(hidden_channels // 2, num_clusters)
)

def forward(self, data: HeteroData) -> Tuple[torch.Tensor, Dict[str,
torch.Tensor]]:
    """Forward pass through the network."""
    # Encode node features
    x_dict = {
        'alert': self.alert_encoder(data['alert'].x),
        'user': self.user_encoder(data['user'].x),
        'host': self.host_encoder(data['host'].x),
        'ip': self.ip_encoder(data['ip'].x)
    }

    # Message passing through GAT layers
    for conv in self.convs:
        x_dict = conv(x_dict, data.edge_index_dict)
        x_dict = {key: F.elu(x) for key, x in x_dict.items()}

    # Global pooling over alert nodes
    alert_emb = x_dict['alert']
    pooled_emb = global_mean_pool(alert_emb, torch.zeros(alert_emb.size(0),
dtype=torch.long, device=alert_emb.device))

    # Classification
    cluster_logits = self.classifier(pooled_emb)

    return cluster_logits, x_dict

```

Stage 5: Post-Processing

Cluster Refinement, Attack Chain Extraction

File: `core/postprocessing.py`

```

import pandas as pd
import numpy as np
from collections import defaultdict
from typing import List, Dict, Tuple, Optional
import networkx as nx
from datetime import datetime, timedelta

class ClusterPostProcessor:
    """
    Post-processing pipeline for cluster refinement and attack chain extraction.

    Operations:
    1. Cluster cleaning (remove singletons, merge small clusters)
    2. Attack chain extraction (temporal progression analysis)
    3. MITRE ATT&CK tactic mapping
    4. Severity assessment
    """

    def __init__(self, min_cluster_size: int = 2, max_cluster_size: int = 50):
        self.min_cluster_size = min_cluster_size
        self.max_cluster_size = max_cluster_size
        self.tactic_map = self._load_tactic_map()

    def postprocess_clusters(self, df: pd.DataFrame) -> pd.DataFrame:
        """Main post-processing pipeline."""
        print(f"Post-processing {df['cluster'].nunique()} clusters")

        # Step 1: Cluster cleaning
        df = self._clean_clusters(df)

        # Step 2: Attack chain extraction
        df = self._extract_attack_chains(df)

        # Step 3: Tactic mapping
        df = self._map_tactics(df)

        # Step 4: Severity assessment
        df = self._assess_severity(df)

        # Step 5: Cluster ranking
        df = self._rank_clusters(df)

        print(f"Post-processing complete. Final clusters: {df['cluster'].nunique()}")
        return df

    def _clean_clusters(self, df: pd.DataFrame) -> pd.DataFrame:
        """Clean and refine clusters."""
        cleaned_df = df.copy()

        # Remove singletons (clusters with only one alert)
        cluster_sizes = df['cluster'].value_counts()
        singletons = cluster_sizes[cluster_sizes < self.min_cluster_size].index

```

```

        cleaned_df = cleaned_df[~cleaned_df['cluster'].isin(singletons)]

        # Merge very small clusters with similar ones
        cleaned_df = self._merge_small_clusters(cleaned_df)

        # Split oversized clusters
        cleaned_df = self._split_large_clusters(cleaned_df)

    return cleaned_df

def _merge_small_clusters(self, df: pd.DataFrame) -> pd.DataFrame:
    """Merge small clusters with similar characteristics."""
    cluster_sizes = df['cluster'].value_counts()
    small_clusters = cluster_sizes[cluster_sizes < 5].index

    if len(small_clusters) == 0:
        return df

    # Find similar clusters based on common entities
    merged_df = df.copy()

    for cluster_id in small_clusters:
        cluster_data = df[df['cluster'] == cluster_id]

        # Find most similar cluster
        similar_cluster = self._find_most_similar_cluster(cluster_data, df,
cluster_id)

        if similar_cluster is not None:
            # Merge clusters
            merged_df.loc[merged_df['cluster'] == cluster_id, 'cluster'] =
similar_cluster
            print(f"Merged cluster {cluster_id} into {similar_cluster}")

    return merged_df

def _find_most_similar_cluster(
    self,
    cluster_data: pd.DataFrame,
    df: pd.DataFrame,
    exclude_cluster: int
) -> Optional[int]:
    """Find most similar cluster based on shared entities."""
    cluster_entities = self._extract_cluster_entities(cluster_data)

    best_cluster = None
    best_similarity = 0.0

    for other_cluster_id in df['cluster'].unique():
        if other_cluster_id == exclude_cluster:
            continue

        other_cluster_data = df[df['cluster'] == other_cluster_id]
        other_entities = self._extract_cluster_entities(other_cluster_data)

```

```

        # Calculate Jaccard similarity
        similarity = self._jaccard_similarity(cluster_entities,
other_entities)

        if similarity > best_similarity and similarity > 0.3: # Threshold for
similarity
            best_similarity = similarity
            best_cluster = other_cluster_id

    return best_cluster

def _extract_cluster_entities(self, cluster_data: pd.DataFrame) -> set:
    """Extract unique entities from cluster."""
    entities = set()

    # Add IP addresses
    for col in ['SourceAddress', 'DestinationAddress', 'DeviceAddress']:
        if col in cluster_data.columns:
            entities.update(cluster_data[col].dropna().unique())

    # Add usernames
    for col in ['SourceUserName', 'DestinationUserName']:
        if col in cluster_data.columns:
            entities.update(cluster_data[col].dropna().unique())

    # Add hostnames
    for col in ['SourceHostName', 'DestinationHostName', 'DeviceHostName']:
        if col in cluster_data.columns:
            entities.update(cluster_data[col].dropna().unique())

    return entities

def _jaccard_similarity(self, set1: set, set2: set) -> float:
    """Calculate Jaccard similarity between two sets."""
    intersection = len(set1.intersection(set2))
    union = len(set1.union(set2))
    return intersection / union if union > 0 else 0.0

def _split_large_clusters(self, df: pd.DataFrame) -> pd.DataFrame:
    """Split oversized clusters based on temporal or entity separation."""
    cluster_sizes = df['cluster'].value_counts()
    large_clusters = cluster_sizes[cluster_sizes >
self.max_cluster_size].index

    if len(large_clusters) == 0:
        return df

    split_df = df.copy()
    next_cluster_id = df['cluster'].max() + 1

    for cluster_id in large_clusters:
        cluster_data = df[df['cluster'] == cluster_id]

```



```

        # Try splitting by time gaps
        sub_clusters = self._split_by_time_gaps(cluster_data)

        if len(sub_clusters) > 1:
            # Assign new cluster IDs
            for i, sub_cluster_indices in enumerate(sub_clusters[1:], 1):
                split_df.loc[sub_cluster_indices, 'cluster'] = next_cluster_id
                next_cluster_id += 1

            print(f"Split cluster {cluster_id} into {len(sub_clusters)} sub-
clusters")

        return split_df

def _split_by_time_gaps(self, cluster_data: pd.DataFrame) -> List[pd.Index]:
    """Split cluster by significant time gaps."""
    if 'timestamp' not in cluster_data.columns:
        return [cluster_data.index]

    # Sort by timestamp
    sorted_data = cluster_data.sort_values('timestamp')

    # Find time gaps > 24 hours
    time_gaps = []
    for i in range(1, len(sorted_data)):
        time_diff = sorted_data.iloc[i]['timestamp'] - sorted_data.iloc[i-1]
        if time_diff > timedelta(hours=24):
            time_gaps.append(i)

    # Split at gaps
    sub_clusters = []
    start_idx = 0

    for gap_idx in time_gaps:
        sub_clusters.append(sorted_data.iloc[start_idx:gap_idx].index)
        start_idx = gap_idx

    sub_clusters.append(sorted_data.iloc[start_idx:].index)

    return sub_clusters

def _extract_attack_chains(self, df: pd.DataFrame) -> pd.DataFrame:
    """Extract attack progression chains for each cluster."""
    chain_df = df.copy()
    chain_df['attack_chain'] = None
    chain_df['chain_stage'] = None

    for cluster_id in df['cluster'].unique():
        cluster_data = df[df['cluster'] == cluster_id]

        # Sort by timestamp
        if 'timestamp' in cluster_data.columns:
            sorted_cluster = cluster_data.sort_values('timestamp')

```

```

        # Extract attack chain
        attack_chain = self._build_attack_chain(sorted_cluster)

        # Assign chain information
        for i, (idx, row) in enumerate(sorted_cluster.iterrows()):
            chain_df.loc[idx, 'attack_chain'] = f"chain_{cluster_id}"
            chain_df.loc[idx, 'chain_stage'] = i + 1

    return chain_df

def _build_attack_chain(self, sorted_cluster: pd.DataFrame) -> List[Dict]:
    """Build attack chain from sorted alerts."""
    chain = []

    for _, alert in sorted_cluster.iterrows():
        chain_stage = {
            'timestamp': alert.get('timestamp'),
            'alert_type': alert.get('MalwareIntelAttackType', 'unknown'),
            'tactic':
self._map_alert_to_tactic(alert.get('MalwareIntelAttackType', '')),
            'severity': alert.get('AttackSeverity', 'medium'),
            'src_ip': alert.get('SourceAddress', ''),
            'dst_ip': alert.get('DestinationAddress', ''),
            'user': alert.get('SourceUserName', '')
        }
        chain.append(chain_stage)

    return chain

def _map_tactics(self, df: pd.DataFrame) -> pd.DataFrame:
    """Map alert types to MITRE ATT&CK tactics."""
    tactic_df = df.copy()
    tactic_df['mitre_tactic'] = tactic_df['MalwareIntelAttackType'].apply(
        lambda x: self._map_alert_to_tactic(x)
    )
    return tactic_df

def _map_alert_to_tactic(self, alert_type: str) -> str:
    """Map specific alert type to MITRE ATT&CK tactic."""
    if pd.isna(alert_type):
        return 'UNKNOWN'

    alert_type_lower = alert_type.lower()

    # Simple keyword-based mapping
    tactic_keywords = {
        'RECONNAISSANCE': ['scan', 'probe', 'discover', 'enum', 'recon'],
        'INITIAL ACCESS': ['phish', 'login', 'auth', 'credential', 'brute'],
        'EXECUTION': ['execute', 'run', 'command', 'script', 'payload'],
        'PERSISTENCE': ['install', 'service', 'registry', 'startup',
'scheduled'],
        'PRIVILEGE ESCALATION': ['sudo', 'admin', 'privilege', 'escalate',
'root'],
    }

```

```

        'DEFENSE EVASION': ['obfuscate', 'encode', 'encrypt', 'hide',
                              'stealth'],
        'CREDENTIAL ACCESS': ['dump', 'password', 'hash', 'credential',
                              'theft'],
        'DISCOVERY': ['enum', 'find', 'locate', 'search', 'query'],
        'LATERAL MOVEMENT': ['remote', 'ssh', 'rdp', 'smb', 'lateral'],
        'COLLECTION': ['exfil', 'copy', 'transfer', 'collect', 'gather'],
        'COMMAND AND CONTROL': ['c2', 'cnc', 'beacon', 'callback', 'command'],
        'IMPACT': ['delete', 'encrypt', 'damage', 'destroy', 'impact']
    }

    for tactic, keywords in tactic_keywords.items():
        if any(keyword in alert_type_lower for keyword in keywords):
            return tactic

    return 'UNKNOWN'

def _assess_severity(self, df: pd.DataFrame) -> pd.DataFrame:
    """Assess cluster severity based on alert types and tactics."""
    severity_df = df.copy()
    severity_df['cluster_severity'] = 'medium'

    for cluster_id in df['cluster'].unique():
        cluster_data = df[df['cluster'] == cluster_id]

        # Calculate severity score
        severity_score = self._calculate_severity_score(cluster_data)

        # Map score to severity level
        if severity_score >= 8:
            severity_level = 'critical'
        elif severity_score >= 6:
            severity_level = 'high'
        elif severity_score >= 4:
            severity_level = 'medium'
        else:
            severity_level = 'low'

        severity_df.loc[severity_df['cluster'] == cluster_id,
                        'cluster_severity'] = severity_level

    return severity_df

def _calculate_severity_score(self, cluster_data: pd.DataFrame) -> float:
    """Calculate severity score for cluster."""
    score = 0.0

    # Base score from alert severities
    severity_weights = {'low': 1, 'medium': 2, 'high': 3, 'critical': 4}
    for _, alert in cluster_data.iterrows():
        alert_severity = alert.get('AttackSeverity', 'medium')
        score += severity_weights.get(alert_severity, 2)

    # Bonus points for high-impact tactics

```

```

high_impact_tactics = [
    'PRIVILEGE ESCALATION', 'CREDENTIAL ACCESS',
    'LATERAL MOVEMENT', 'IMPACT'
]

for _, alert in cluster_data.iterrows():
    tactic = alert.get('mitre_tactic', 'UNKNOWN')
    if tactic in high_impact_tactics:
        score += 2

# Normalize by cluster size
normalized_score = score / len(cluster_data)

return min(normalized_score, 10.0) # Cap at 10

def _rank_clusters(self, df: pd.DataFrame) -> pd.DataFrame:
    """Rank clusters by importance."""
    ranked_df = df.copy()

    # Calculate cluster scores
    cluster_scores = {}
    for cluster_id in df['cluster'].unique():
        cluster_data = df[df['cluster'] == cluster_id]

        # Score based on size, severity, and tactic diversity
        size_score = len(cluster_data) / 10.0 # Normalize size
        severity_score = self._calculate_severity_score(cluster_data)
        tactic_diversity = len(cluster_data['mitre_tactic'].unique())

        total_score = size_score + severity_score + tactic_diversity
        cluster_scores[cluster_id] = total_score

    # Rank clusters
    sorted_clusters = sorted(cluster_scores.items(), key=lambda x: x[1],
reverse=True)
    cluster_ranks = {cluster_id: rank + 1 for rank, (cluster_id, _) in
enumerate(sorted_clusters)}

    # Assign ranks
    for cluster_id, rank in cluster_ranks.items():
        ranked_df.loc[ranked_df['cluster'] == cluster_id, 'cluster_rank'] =
rank

    return ranked_df

def _load_tactic_map(self) -> Dict:
    """Load MITRE ATT&CK tactic mapping."""
    # This would typically load from a JSON file
    return {
        'scan': 'RECONNAISSANCE',
        'phishing': 'INITIAL ACCESS',
        'execution': 'EXECUTION',
        'persistence': 'PERSISTENCE',
        'privilege_escalation': 'PRIVILEGE ESCALATION',

```

```
        'defense_evasion': 'DEFENSE EVASION',
        'credential_access': 'CREDENTIAL ACCESS',
        'discovery': 'DISCOVERY',
        'lateral_movement': 'LATERAL MOVEMENT',
        'collection': 'COLLECTION',
        'command_control': 'COMMAND AND CONTROL',
        'impact': 'IMPACT'
    }
```

Stage 6: Output Generation

JSON Export + Dashboard Visualization

File: `core/output.py`

```
import json
import logging
import os
import pandas as pd
from datetime import datetime
from typing import Dict, List, Any
from pathlib import Path

logger = logging.getLogger("mitre-core.output")

class OutputGenerator:
    """
    Generate comprehensive output from correlation results.

    Output Formats:
    1. JSON export for API consumption
    2. CSV export for analysis
    3. Dashboard visualization data
    4. Summary reports
    """

    def __init__(self, output_dir: str = "outputs"):
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(exist_ok=True)

        # Load tactic map
        self.tactic_map = self._load_tactic_map()

        # Attack stage definitions
        self.attack_stages = {
            "Initial": [
                ['INITIAL ACCESS', 'EXECUTION'],
                ['INITIAL ACCESS', 'EXECUTION', 'PERSISTENCE'],
                ['INITIAL ACCESS', 'CREDENTIAL ACCESS', 'DISCOVERY']
            ],

```

```

        "Partial": [
            ['PERSISTENCE', 'PRIVILEGE ESCALATION', 'CREDENTIAL ACCESS',
'DISCOVERY']
        ],
        "Complete": [
            ['INITIAL ACCESS', 'EXECUTION', 'PERSISTENCE', 'PRIVILEGE
ESCALATION',
            'DEFENSE EVASION', 'CREDENTIAL ACCESS', 'DISCOVERY', 'LATERAL
MOVEMENT',
            'COLLECTION', 'COMMAND AND CONTROL', 'IMPACT'],
            ['INITIAL ACCESS', 'EXECUTION', 'DEFENSE EVASION', 'EXFILTRATION',
'IMPACT'],
            ['PERSISTENCE', 'CREDENTIAL ACCESS', 'COLLECTION', 'EXFILTRATION']
        ]
    }

def generate_comprehensive_output(
    self,
    df: pd.DataFrame,
    correlation_result: 'CorrelationResult',
    output_prefix: str = None
) -> Dict[str, str]:
    """
    Generate all output formats.

    Args:
        df: Clustered alert DataFrame
        correlation_result: Result from correlation pipeline
        output_prefix: Prefix for output files

    Returns:
        Dictionary with paths to generated files
    """
    if output_prefix is None:
        output_prefix =
f"mitre_core_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

    output_files = {}

    # 1. JSON export
    json_file = self.generate_json_output(df, correlation_result,
output_prefix)
    output_files['json'] = json_file

    # 2. CSV export
    csv_file = self.generate_csv_output(df, output_prefix)
    output_files['csv'] = csv_file

    # 3. Dashboard data
    dashboard_file = self.generate_dashboard_data(df, output_prefix)
    output_files['dashboard'] = dashboard_file

    # 4. Summary report
    summary_file = self.generate_summary_report(df, correlation_result,

```

```

output_prefix)
    output_files['summary'] = summary_file

    logger.info(f"Generated {len(output_files)} output files with prefix:
{output_prefix}")
    return output_files

def generate_json_output(
    self,
    df: pd.DataFrame,
    correlation_result: 'CorrelationResult',
    output_prefix: str
) -> str:
    """Generate comprehensive JSON output."""

    # Build cluster information
    clusters = []
    for cluster_id in sorted(df['cluster'].unique()):
        cluster_data = df[df['cluster'] == cluster_id]
        cluster_info = self._build_cluster_info(cluster_data, cluster_id)
        clusters.append(cluster_info)

    # Build summary statistics
    summary = self._build_summary(df, correlation_result)

    # Build attack timeline
    timeline = self._build_attack_timeline(df)

    # Build MITRE ATT&CK coverage
    tactic_coverage = self._build_tactic_coverage(df)

    # Assemble final JSON
    output = {
        "metadata": {
            "generated_at": datetime.now().isoformat(),
            "mitre_core_version": "2.1",
            "correlation_method": correlation_result.method_used,
            "total_runtime_seconds": correlation_result.runtime_seconds,
            "confidence_score": correlation_result.confidence_score
        },
        "summary": summary,
        "clusters": clusters,
        "attack_timeline": timeline,
        "mitre_tactic_coverage": tactic_coverage,
        "performance_metrics": {
            "alerts_processed": len(df),
            "clusters_found": correlation_result.num_clusters,
            "average_cluster_size": len(df) / correlation_result.num_clusters,
            "processing_rate": len(df) / correlation_result.runtime_seconds
        }
    }

    # Save to file
    output_file = self.output_dir / f"{output_prefix}_results.json"

```

```

        with open(output_file, 'w') as f:
            json.dump(output, f, indent=2, default=str)

        return str(output_file)

    def _build_cluster_info(self, cluster_data: pd.DataFrame, cluster_id: int) -> Dict[str, Any]:
        """Build detailed cluster information."""

        # Basic cluster info
        cluster_info = {
            "cluster_id": int(cluster_id),
            "size": len(cluster_data),
            "time_span": self._calculate_time_span(cluster_data),
            "severity": cluster_data['cluster_severity'].iloc[0] if
'cluster_severity' in cluster_data.columns else 'medium',
            "rank": int(cluster_data['cluster_rank'].iloc[0]) if 'cluster_rank' in
cluster_data.columns else None,
            "attack_stage": self._classify_attack_stage(cluster_data),
            "confidence": self._calculate_cluster_confidence(cluster_data)
        }

        # Entity information
        cluster_info["entities"] = self._extract_cluster_entities(cluster_data)

        # Alert breakdown
        cluster_info["alert_breakdown"] =
self._build_alert_breakdown(cluster_data)

        # Attack chain
        if 'attack_chain' in cluster_data.columns:
            cluster_info["attack_chain"] = self._build_attack_chain(cluster_data)

        # MITRE tactics
        if 'mitre_tactic' in cluster_data.columns:
            tactics = cluster_data['mitre_tactic'].value_counts().to_dict()
            cluster_info["mitre_tactics"] = tactics

        return cluster_info

    def _calculate_time_span(self, cluster_data: pd.DataFrame) -> Dict[str, str]:
        """Calculate time span of cluster."""
        if 'timestamp' not in cluster_data.columns:
            return {"start": "unknown", "end": "unknown", "duration": "unknown"}

        timestamps = pd.to_datetime(cluster_data['timestamp'])
        start_time = timestamps.min().isoformat()
        end_time = timestamps.max().isoformat()
        duration = str(timestamps.max() - timestamps.min())

        return {
            "start": start_time,
            "end": end_time,
            "duration": duration
        }

```



```

    }

    def _classify_attack_stage(self, cluster_data: pd.DataFrame) -> str:
        """Classify attack stage based on observed tactics."""
        if 'mitre_tactic' not in cluster_data.columns:
            return "Unknown"

        observed_tactics = set(cluster_data['mitre_tactic'].unique())

        # Check against attack stage definitions
        for stage, tactic_combinations in self.attack_stages.items():
            for combination in tactic_combinations:
                if set(combination).issubset(observed_tactics):
                    return stage

        return "Partial"

    def _calculate_cluster_confidence(self, cluster_data: pd.DataFrame) -> float:
        """Calculate confidence score for cluster."""
        # Simple confidence based on size and tactic diversity
        size_score = min(len(cluster_data) / 10.0, 1.0)

        if 'mitre_tactic' in cluster_data.columns:
            tactic_diversity = len(cluster_data['mitre_tactic'].unique())
            diversity_score = min(tactic_diversity / 5.0, 1.0)
        else:
            diversity_score = 0.5

        confidence = (size_score + diversity_score) / 2.0
        return round(confidence, 3)

    def _extract_cluster_entities(self, cluster_data: pd.DataFrame) -> Dict[str,
List[str]]:
        """Extract unique entities from cluster."""
        entities = {
            "source_ips": [],
            "destination_ips": [],
            "users": [],
            "hosts": []
        }

        # Extract IPs
        for col in ['SourceAddress', 'DestinationAddress']:
            if col in cluster_data.columns:
                entities["source_ips"].extend(cluster_data[col].dropna().unique())

        # Extract users
        if 'SourceUserName' in cluster_data.columns:
            entities["users"].extend(cluster_data['SourceUserName'].dropna().unique())

        # Extract hosts
        for col in ['SourceHostName', 'DestinationHostName']:
            if col in cluster_data.columns:

```

```

        entities["hosts"].extend(cluster_data[col].dropna().unique())

    # Remove duplicates and 'UNKNOWN'
    for key in entities:
        entities[key] = list(set([e for e in entities[key] if e not in
['UNKNOWN', '0.0.0.0']]))

    return entities

def _build_alert_breakdown(self, cluster_data: pd.DataFrame) -> Dict[str,
Any]:
    """Build alert type breakdown."""
    if 'MalwareIntelAttackType' not in cluster_data.columns:
        return {}

    alert_counts =
cluster_data['MalwareIntelAttackType'].value_counts().to_dict()

    return {
        "alert_types": alert_counts,
        "most_common": max(alert_counts.items(), key=lambda x: x[1]) if
alert_counts else None,
        "diversity_score": len(alert_counts) / len(cluster_data) if
len(cluster_data) > 0 else 0
    }

def _build_attack_chain(self, cluster_data: pd.DataFrame) -> List[Dict[str,
Any]]:
    """Build attack chain from cluster data."""
    if 'timestamp' not in cluster_data.columns:
        return []

    # Sort by timestamp
    sorted_cluster = cluster_data.sort_values('timestamp')

    chain = []
    for _, alert in sorted_cluster.iterrows():
        chain_step = {
            "timestamp": alert['timestamp'].isoformat() if
pd.notna(alert['timestamp']) else None,
            "alert_type": alert.get('MalwareIntelAttackType', 'unknown'),
            "tactic": alert.get('mitre_tactic', 'unknown'),
            "severity": alert.get('AttackSeverity', 'medium'),
            "source_ip": alert.get('SourceAddress', ''),
            "destination_ip": alert.get('DestinationAddress', ''),
            "user": alert.get('SourceUserName', ''),
            "description": f"{alert.get('MalwareIntelAttackType', 'unknown')}"
            alert"
        }
        chain.append(chain_step)

    return chain

def _build_summary(self, df: pd.DataFrame, correlation_result:

```

```

'CorrelationResult') -> Dict[str, Any]:
    """Build summary statistics."""

    # Cluster size distribution
    cluster_sizes = df['cluster'].value_counts()

    # Severity distribution
    severity_dist = df['cluster_severity'].value_counts().to_dict() if
'cluster_severity' in df.columns else {}

    # Tactic distribution
    tactic_dist = df['mitre_tactic'].value_counts().to_dict() if
'mitre_tactic' in df.columns else {}

    return {
        "total_alerts": len(df),
        "total_clusters": correlation_result.num_clusters,
        "correlation_method": correlation_result.method_used,
        "runtime_seconds": correlation_result.runtime_seconds,
        "average_cluster_size": round(len(df) /
correlation_result.num_clusters, 2),
        "cluster_size_distribution": {
            "min": int(cluster_sizes.min()),
            "max": int(cluster_sizes.max()),
            "median": float(cluster_sizes.median()),
            "mean": float(cluster_sizes.mean())
        },
        "severity_distribution": severity_dist,
        "top_tactics": dict(list(tactic_dist.items())[:5]) if tactic_dist else
{},
        "confidence_score": correlation_result.confidence_score
    }

def _build_attack_timeline(self, df: pd.DataFrame) -> List[Dict[str, Any]]:
    """Build attack timeline across all clusters."""
    if 'timestamp' not in df.columns:
        return []

    # Sort all alerts by timestamp
    sorted_df = df.sort_values('timestamp')

    timeline = []
    for _, alert in sorted_df.iterrows():
        timeline_event = {
            "timestamp": alert['timestamp'].isoformat(),
            "cluster_id": int(alert['cluster']),
            "alert_type": alert.get('MalwareIntelAttackType', 'unknown'),
            "tactic": alert.get('mitre_tactic', 'unknown'),
            "severity": alert.get('AttackSeverity', 'medium'),
            "source_ip": alert.get('SourceAddress', ''),
            "destination_ip": alert.get('DestinationAddress', ''),
            "user": alert.get('SourceUserName', '')
        }
        timeline.append(timeline_event)

```

```

        return timeline

def _build_tactic_coverage(self, df: pd.DataFrame) -> Dict[str, Any]:
    """Build MITRE ATT&CK tactic coverage analysis."""
    if 'mitre_tactic' not in df.columns:
        return {}

    # Count tactics
    tactic_counts = df['mitre_tactic'].value_counts().to_dict()

    # Calculate coverage percentages
    total_tactics = len(self.tactic_map)
    observed_tactics = len(tactic_counts)
    coverage_percentage = (observed_tactics / total_tactics) * 100 if
total_tactics > 0 else 0

    return {
        "total_mitre_tactics": total_tactics,
        "observed_tactics": observed_tactics,
        "coverage_percentage": round(coverage_percentage, 2),
        "tactic_frequency": tactic_counts,
        "missing_tactics": list(set(self.tactic_map.values()) -
set(tactic_counts.keys()))
    }

def generate_csv_output(self, df: pd.DataFrame, output_prefix: str) -> str:
    """Generate CSV export for analysis."""
    csv_file = self.output_dir / f"{output_prefix}_clusters.csv"
    df.to_csv(csv_file, index=False)
    return str(csv_file)

def generate_dashboard_data(self, df: pd.DataFrame, output_prefix: str) ->
str:
    """Generate data for dashboard visualization."""

    dashboard_data = {
        "cluster_stats": self._generate_cluster_stats(df),
        "tactic_distribution": self._generate_tactic_distribution(df),
        "timeline_data": self._generate_timeline_data(df),
        "severity_breakdown": self._generate_severity_breakdown(df),
        "entity_network": self._generate_entity_network(df)
    }

    dashboard_file = self.output_dir / f"{output_prefix}_dashboard.json"
    with open(dashboard_file, 'w') as f:
        json.dump(dashboard_data, f, indent=2, default=str)

    return str(dashboard_file)

def _generate_cluster_stats(self, df: pd.DataFrame) -> Dict[str, Any]:
    """Generate cluster statistics for dashboard."""
    cluster_sizes = df['cluster'].value_counts()

```

```

        return {
            "total_clusters": len(cluster_sizes),
            "cluster_sizes": cluster_sizes.to_dict(),
            "size_histogram": [
                {"size_range": f"{i}-{i+9}", "count": ((cluster_sizes >= i) &
(cluster_sizes < i+10)).sum()}
                for i in range(1, 51, 10)
            ]
        }

def _generate_tactic_distribution(self, df: pd.DataFrame) -> Dict[str, Any]:
    """Generate tactic distribution for dashboard."""
    if 'mitre_tactic' not in df.columns:
        return {}

    tactic_counts = df['mitre_tactic'].value_counts()

    return {
        "tactics": tactic_counts.to_dict(),
        "total_alerts": len(df)
    }

def _generate_timeline_data(self, df: pd.DataFrame) -> List[Dict[str, Any]]:
    """Generate timeline data for dashboard."""
    if 'timestamp' not in df.columns:
        return []

    # Group by hour
    df['hour'] = pd.to_datetime(df['timestamp']).dt.floor('H')
    hourly_counts = df.groupby('hour').size().reset_index(name='alert_count')

    timeline = []
    for _, row in hourly_counts.iterrows():
        timeline.append({
            "timestamp": row['hour'].isoformat(),
            "alert_count": int(row['alert_count']),
            "cluster_count": df[df['timestamp'].dt.floor('H') == row['hour']]
['cluster'].nunique()
        })

    return timeline

def _generate_severity_breakdown(self, df: pd.DataFrame) -> Dict[str, Any]:
    """Generate severity breakdown for dashboard."""
    if 'cluster_severity' not in df.columns:
        return {}

    severity_counts = df['cluster_severity'].value_counts()

    return {
        "severity_distribution": severity_counts.to_dict(),
        "severity_by_cluster": df.groupby('cluster')
['cluster_severity'].first().value_counts().to_dict()
    }

```

```

def _generate_entity_network(self, df: pd.DataFrame) -> Dict[str, Any]:
    """Generate entity network data for dashboard."""
    nodes = []
    edges = []

    # Add cluster nodes
    for cluster_id in df['cluster'].unique():
        cluster_data = df[df['cluster'] == cluster_id]
        nodes.append({
            "id": f"cluster_{cluster_id}",
            "type": "cluster",
            "label": f"Cluster {cluster_id}",
            "size": len(cluster_data),
            "severity": cluster_data['cluster_severity'].iloc[0] if
'cluster_severity' in cluster_data.columns else 'medium'
        })

    # Add entity nodes and edges
    entity_types = ['SourceAddress', 'DestinationAddress', 'SourceUserName',
'SourceHostName']

    for entity_type in entity_types:
        if entity_type not in df.columns:
            continue

        for entity in df[entity_type].dropna().unique():
            if entity in ['UNKNOWN', '0.0.0.0']:
                continue

            # Add entity node
            nodes.append({
                "id": f"{entity_type}_{entity}",
                "type": entity_type.lower().replace('source',
'').replace('destination', '').replace('name', ''),
                "label": str(entity),
                "size": 5
            })

            # Add edges to clusters
            clusters_with_entity = df[df[entity_type] == entity]
['cluster'].unique()
            for cluster_id in clusters_with_entity:
                edges.append({
                    "source": f"{entity_type}_{entity}",
                    "target": f"cluster_{cluster_id}",
                    "weight": df[(df[entity_type] == entity) & (df['cluster']
== cluster_id)].shape[0]
                })

    return {
        "nodes": nodes,
        "edges": edges
    }

```

```

def generate_summary_report(
    self,
    df: pd.DataFrame,
    correlation_result: 'CorrelationResult',
    output_prefix: str
) -> str:
    """Generate human-readable summary report."""

    report_lines = [
        "# MITRE-CORE Analysis Report",
        f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
        "",
        "## Summary",
        f"- Total Alerts Processed: {len(df):,}",
        f"- Clusters Identified: {correlation_result.num_clusters}",
        f"- Correlation Method: {correlation_result.method_used}",
        f"- Processing Time: {correlation_result.runtime_seconds:.2f}
seconds",
        f"- Average Cluster Size: {len(df) /
correlation_result.num_clusters:.1f}",
        "",
        "## Top Clusters by Severity"
    ]

    # Add top clusters
    if 'cluster_severity' in df.columns and 'cluster_rank' in df.columns:
        top_clusters = df.nlargest(10, 'cluster_rank')['cluster'].unique()
        for cluster_id in top_clusters[:5]:
            cluster_data = df[df['cluster'] == cluster_id]
            severity = cluster_data['cluster_severity'].iloc[0]
            size = len(cluster_data)

            report_lines.extend([
                f"### Cluster {cluster_id} ({severity})",
                f"- Size: {size} alerts",
                f"- Time Span: {self._calculate_time_span(cluster_data)
['duration']}",
                ""
            ])

    # Add MITRE ATT&CK coverage
    if 'mitre_tactic' in df.columns:
        tactic_counts = df['mitre_tactic'].value_counts()
        report_lines.extend([
            "## MITRE ATT&CK Tactics Observed",
            ""
        ])

        for tactic, count in tactic_counts.head(10).items():
            report_lines.append(f"- {tactic}: {count} alerts")

    # Save report
    report_file = self.output_dir / f"{output_prefix}_report.md"

```

```

        with open(report_file, 'w') as f:
            f.write('\n'.join(report_lines))

        return str(report_file)

def _load_tactic_map(self) -> Dict:
    """Load MITRE ATT&CK tactic mapping."""
    tactic_map_path = Path(__file__).parent.parent / "tactic_map.json"

    try:
        with open(tactic_map_path, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        logger.warning(f"Tactic map not found at {tactic_map_path}")
        return {}
    except json.JSONDecodeError as e:
        logger.error(f"Tactic map malformed: {e}")
        return {}

```

Three Correlation Methods

1. Union-Find (Baseline)

File: `core/correlation_indexer.py`

```

from collections import defaultdict
import numpy as np
import pandas as pd
from typing import Dict, List, Tuple, Optional

def weighted_correlation_score(
    addresses_common: set,
    usernames_common: set,
    temporal_proximity: float = 0,
    n_address_cols: int = 1,
    n_username_cols: int = 1,
    use_temporal: bool = False,
) -> float:
    """
    Compute normalized alert correlation score in [0, 1].

    The raw score (addresses * 0.6 + usernames * 0.3 + temporal * 0.1) is divided
    by the theoretical maximum for the given column configuration so the result is
    always in [0, 1], making it directly comparable to
    confidence_guided_threshold()
    outputs which are also in [0.1, 0.9].
    """

    # Weight components
    address_weight = 0.6
    username_weight = 0.3

```



```

temporal_weight = 0.1 if use_temporal else 0.0

# Calculate raw score
raw_score = (
    len(addresses_common) * address_weight +
    len(usernames_common) * username_weight +
    temporal_proximity * temporal_weight
)

# Calculate theoretical maximum score
max_score = (
    n_address_cols * address_weight +
    n_username_cols * username_weight +
    temporal_weight
)

# Normalize to [0, 1]
normalized_score = raw_score / max_score if max_score > 0 else 0.0

return normalized_score

class UnionFindClusterer:
    """
    Union-Find (Disjoint Set) based clustering algorithm.

    Complexity:  $O(n \alpha(n))$  - nearly linear time
    Best for: Small datasets (<100 events), real-time processing
    No training required
    """

    def __init__(self, threshold: float = 0.3):
        self.threshold = threshold
        self.parent = {}
        self.rank = {}

    def find(self, x):
        """Find root of element x with path compression."""
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        """Union sets containing x and y."""
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return

        # Union by rank
        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        elif self.rank[root_x] > self.rank[root_y]:
            self.parent[root_y] = root_x

```

```

        else:
            self.parent[root_y] = root_x
            self.rank[root_x] += 1

    def cluster_alerts(self, df: pd.DataFrame, usernames: List[str], addresses:
List[str]) -> pd.DataFrame:
        """
        Cluster alerts using Union-Find algorithm.

        Args:
            df: DataFrame with security alerts
            usernames: List of username column names
            addresses: List of IP address column names

        Returns:
            DataFrame with cluster assignments
        """
        # Initialize Union-Find structure
        n_alerts = len(df)
        for i in range(n_alerts):
            self.parent[i] = i
            self.rank[i] = 0

        # Build similarity graph and union similar alerts
        for i in range(n_alerts):
            for j in range(i + 1, n_alerts):
                score = self._calculate_similarity_score(df.iloc[i], df.iloc[j],
usernames, addresses)

                if score >= self.threshold:
                    self.union(i, j)

        # Extract clusters
        clusters = defaultdict(list)
        for i in range(n_alerts):
            root = self.find(i)
            clusters[root].append(i)

        # Assign cluster IDs
        result_df = df.copy()
        result_df['cluster'] = 0

        for cluster_id, alert_indices in enumerate(clusters.values(), 1):
            for idx in alert_indices:
                result_df.iloc[idx, result_df.columns.get_loc('cluster')] =
cluster_id

        return result_df

    def _calculate_similarity_score(
        self,
        alert1: pd.Series,
        alert2: pd.Series,
        usernames: List[str],

```

```

        addresses: List[str]
    ) -> float:
        """Calculate similarity score between two alerts."""

        # Find common addresses
        common_addresses = set()
        for addr_col in addresses:
            addr1 = alert1.get(addr_col, '')
            addr2 = alert2.get(addr_col, '')
            if addr1 and addr2 and addr1 == addr2 and addr1 != '0.0.0.0':
                common_addresses.add(addr1)

        # Find common usernames
        common_usernames = set()
        for user_col in usernames:
            user1 = alert1.get(user_col, '')
            user2 = alert2.get(user_col, '')
            if user1 and user2 and user1 == user2 and user1 != 'UNKNOWN':
                common_usernames.add(user1)

        # Calculate temporal proximity
        temporal_proximity = 0.0
        if 'timestamp' in alert1 and 'timestamp' in alert2:
            try:
                time1 = pd.to_datetime(alert1['timestamp'])
                time2 = pd.to_datetime(alert2['timestamp'])
                time_diff = abs((time1 - time2).total_seconds())

                # Temporal proximity decreases with time difference
                if time_diff < 3600: # Within 1 hour
                    temporal_proximity = 1.0 - (time_diff / 3600)
                else:
                    temporal_proximity = 0.0
            except:
                pass

        # Calculate weighted correlation score
        score = weighted_correlation_score(
            addresses_common=common_addresses,
            usernames_common=common_usernames,
            temporal_proximity=temporal_proximity,
            n_address_cols=len(addresses),
            n_username_cols=len(usernames),
            use_temporal=True
        )

        return score

def enhanced_correlation(
    data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str],
    threshold: Optional[float] = None,
    use_adaptive_threshold: bool = True

```

```

) -> pd.DataFrame:
    """
    Enhanced Union-Find correlation with adaptive thresholding.

    Args:
        data: DataFrame with security alerts
        usernames: List of username column names
        addresses: List of IP address column names
        threshold: Manual correlation threshold (overrides adaptive)
        use_adaptive_threshold: Use adaptive threshold based on data
characteristics

    Returns:
        DataFrame with cluster assignments
    """
    # Calculate adaptive threshold if needed
    if threshold is None and use_adaptive_threshold:
        threshold = calculate_adaptive_threshold(data, usernames, addresses)
    elif threshold is None:
        threshold = 0.3 # Default threshold

    # Initialize clusterer
    clusterer = UnionFindClusterer(threshold=threshold)

    # Run clustering
    result_df = clusterer.cluster_alerts(data, usernames, addresses)

    return result_df

def calculate_adaptive_threshold(
    df: pd.DataFrame,
    usernames: List[str],
    addresses: List[str]
) -> float:
    """
    Calculate adaptive correlation threshold based on data characteristics.

    Args:
        df: DataFrame with security alerts
        usernames: List of username column names
        addresses: List of IP address column names

    Returns:
        Adaptive threshold value in [0.1, 0.9]
    """
    # Data characteristics
    n_alerts = len(df)

    # Entity diversity metrics
    unique_ips = set()
    unique_users = set()

    for addr_col in addresses:
        if addr_col in df.columns:

```

```

        unique_ips.update(df[addr_col].dropna().unique())

    for user_col in usernames:
        if user_col in df.columns:
            unique_users.update(df[user_col].dropna().unique())

    ip_diversity = len(unique_ips) / n_alerts
    user_diversity = len(unique_users) / n_alerts

    # Adaptive threshold calculation
    # Higher diversity -> lower threshold (more permissive clustering)
    # Lower diversity -> higher threshold (stricter clustering)

    base_threshold = 0.3

    # Adjust based on diversity
    diversity_factor = (ip_diversity + user_diversity) / 2
    diversity_adjustment = (1.0 - diversity_factor) * 0.2

    # Adjust based on dataset size
    if n_alerts < 50:
        size_adjustment = 0.1 # More permissive for small datasets
    elif n_alerts > 1000:
        size_adjustment = -0.1 # Stricter for large datasets
    else:
        size_adjustment = 0.0

    # Calculate final threshold
    adaptive_threshold = base_threshold + diversity_adjustment + size_adjustment

    # Clamp to valid range
    adaptive_threshold = max(0.1, min(0.9, adaptive_threshold))

    return adaptive_threshold

```

2. HGNN (Deep Learning)

File: hgnn/hgnn_correlation.py (continued)

```

class HGNNCorrelationEngine:
    """
    Heterogeneous Graph Neural Network correlation engine.

    Architecture: Heterogeneous Graph Attention Network
    Node Types: alert, user, host, ip entities
    Edge Types: ownership, involvement, temporal proximity
    Training: Contrastive pre-training + supervised fine-tuning
    Accuracy: 86.45% on campaign prediction
    Best for: Large datasets (>1000 events)
    """

```

```

def __init__(
    self,
    model_path: str,
    device: str = 'cpu',
    confidence_threshold: float = 0.6
):
    self.model_path = model_path
    self.device = device
    self.confidence_threshold = confidence_threshold

    # Load trained model
    self.model = self._load_model()
    self.model.eval()

    # Initialize graph converter
    self.graph_converter = AlertToGraphConverter(device=device)

    # Confidence gate for UF refinement
    self.CONFIDENCE_GATE = confidence_threshold

def _load_model(self) -> MITREHeteroGNN:
    """Load trained HGNN model."""
    model = MITREHeteroGNN(
        hidden_channels=64,
        num_heads=4,
        num_layers=2,
        num_clusters=50 # Adjustable based on expected clusters
    )

    checkpoint = torch.load(self.model_path, map_location=self.device)
    model.load_state_dict(checkpoint['model_state_dict'])
    model.to(self.device)

    return model

def correlate(
    self,
    data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str],
    confidence_threshold: Optional[float] = None
) -> pd.DataFrame:
    """
    Run HGNN correlation analysis.

    Args:
        data: Preprocessed alert DataFrame
        usernames: List of username column names
        addresses: List of IP address column names
        confidence_threshold: Override default confidence threshold

    Returns:
        DataFrame with cluster assignments and confidence scores
    """

```

```

        if confidence_threshold is not None:
            self.confidence_threshold = confidence_threshold

        # Convert to heterogeneous graph
        graph_data = self.graph_converter.convert_to_graph(data)

        # Run inference
        with torch.no_grad():
            cluster_logits, node_embeddings = self.model(graph_data)

            # Get cluster assignments and confidence scores
            cluster_probs = F.softmax(cluster_logits, dim=1)
            max_probs, cluster_assignments = torch.max(cluster_probs, dim=1)

            # Assign clusters to alerts
            alert_clusters = cluster_assignments[:len(data)].cpu().numpy()
            alert_confidences = max_probs[:len(data)].cpu().numpy()

        # Create result DataFrame
        result_df = data.copy()
        result_df['cluster'] = alert_clusters + 1 # Start clusters from 1
        result_df['confidence'] = alert_confidences
        result_df['method'] = 'hgnn'

        # Handle low-confidence alerts with Union-Find refinement
        low_confidence_mask = alert_confidences < self.CONFIDENCE_GATE
        if low_confidence_mask.any():
            result_df = self._uf_refinement_pass(result_df, low_confidence_mask,
            usernames, addresses)

        return result_df

    def _uf_refinement_pass(
        self,
        result_df: pd.DataFrame,
        low_confidence_mask: np.ndarray,
        usernames: List[str],
        addresses: List[str]
    ) -> pd.DataFrame:
        """
        Refine low-confidence alerts using Union-Find correlation.

        Args:
            result_df: DataFrame with initial HGNN results
            low_confidence_mask: Boolean mask of low-confidence alerts
            usernames: List of username column names
            addresses: List of IP address column names

        Returns:
            DataFrame with refined cluster assignments
        """
        print(f"Running UF refinement for {low_confidence_mask.sum()} low-
confidence alerts")

```

```

# Extract low-confidence alerts
low_conf_df = result_df[low_confidence_mask].copy()

# Run Union-Find on low-confidence alerts
from core.correlation_indexer import enhanced_correlation

# Use confidence-guided threshold
avg_confidence = result_df.loc[low_confidence_mask, 'confidence'].mean()
uf_threshold = max(0.1, avg_confidence * 0.8) # Slightly more permissive

uf_result = enhanced_correlation(
    low_conf_df,
    usernames,
    addresses,
    threshold=uf_threshold,
    use_adaptive_threshold=False
)

# Merge UF results back
refined_df = result_df.copy()

# Assign new cluster IDs to UF-refined alerts
max_existing_cluster = result_df['cluster'].max()
uf_result['cluster'] = uf_result['cluster'] + max_existing_cluster

# Update low-confidence alerts with UF results
refined_df.loc[low_confidence_mask, 'cluster'] =
uf_result['cluster'].values
refined_df.loc[low_confidence_mask, 'method'] = 'hgnn_uf_refined'

return refined_df

def get_attention_weights(self, data: pd.DataFrame) -> Dict[str,
torch.Tensor]:
    """
    Extract attention weights for explainability.

    Args:
        data: Alert DataFrame

    Returns:
        Dictionary of attention weights per edge type
    """
    # Convert to graph
    graph_data = self.graph_converter.convert_to_graph(data)

    # Hook attention weights
    attention_weights = {}

    def hook_fn(module, input, output):
        if hasattr(module, 'attention_weights'):
            for edge_type, weights in module.attention_weights.items():
                attention_weights[edge_type] = weights.detach()

```



```

# Register hooks
for conv in self.model.convs:
    conv.register_forward_hook(hook_fn)

# Forward pass
with torch.no_grad():
    self.model(graph_data)

return attention_weights

def get_node_embeddings(self, data: pd.DataFrame) -> Dict[str, np.ndarray]:
    """
    Extract node embeddings for analysis.

    Args:
        data: Alert DataFrame

    Returns:
        Dictionary of node embeddings per node type
    """
    # Convert to graph
    graph_data = self.graph_converter.convert_to_graph(data)

    # Forward pass to get embeddings
    with torch.no_grad():
        _, node_embeddings = self.model(graph_data)

    # Convert to numpy arrays
    embeddings = {}
    for node_type, tensor in node_embeddings.items():
        embeddings[node_type] = tensor.cpu().numpy()

    return embeddings

```

3. Hybrid (Ensemble)

File: hgnn/hgnn_integration.py

```

import pandas as pd
import numpy as np
import torch
from typing import List, Dict, Tuple, Optional
import logging
from pathlib import Path

from core.correlation_indexer import enhanced_correlation as
union_find_correlation
from hgnn.hgnn_correlation import HGNNCorrelationEngine, AlertToGraphConverter

logger = logging.getLogger("mitre-core.hgnn_integration")

```

```

class HybridCorrelationEngine:
    """
    Hybrid correlation engine combining Union-Find and HGNN.

    Approach:
    1. Run Union-Find for initial clustering
    2. Run HGNN for refined clustering
    3. Combine results:  $0.7 \times \text{HGNN scores} + 0.3 \times \text{Union-Find scores}$ 

    Best for: Medium datasets (100-1000 events)
    """

    def __init__(
        self,
        hgnn_model_path: str,
        hgnn_weight: float = 0.7,
        union_find_weight: float = 0.3,
        device: str = 'cuda' if torch.cuda.is_available() else 'cpu',
        confidence_threshold: float = 0.6
    ):
        self.hgnn_weight = hgnn_weight
        self.union_find_weight = union_find_weight
        self.device = device
        self.confidence_threshold = confidence_threshold

        # Initialize HGNN engine
        self.hgnn_engine = HGNNCorrelationEngine(
            model_path=hgnn_model_path,
            device=device,
            confidence_threshold=confidence_threshold
        )

        logger.info(f"Hybrid engine initialized: HGNN weight={hgnn_weight}, UF weight={union_find_weight}")

    def correlate(
        self,
        data: pd.DataFrame,
        usernames: List[str],
        addresses: List[str]
    ) -> pd.DataFrame:
        """
        Run hybrid correlation analysis.

        Args:
            data: DataFrame with security alerts
            usernames: List of username column names
            addresses: List of IP address column names

        Returns:
            DataFrame with hybrid cluster assignments
        """
        logger.info(f"Running hybrid correlation on {len(data)} alerts")

```

```

# Step 1: Run Union-Find
logger.info("Step 1: Running Union-Find correlation")
uf_result = self._run_union_find(data, usernames, addresses)

# Step 2: Run HGNN
logger.info("Step 2: Running HGNN correlation")
hgnn_result = self._run_hgnn(data, usernames, addresses)

# Step 3: Combine results
logger.info("Step 3: Combining Union-Find and HGNN results")
hybrid_result = self._combine_results(uf_result, hgnn_result)

# Step 4: Post-processing
logger.info("Step 4: Post-processing hybrid results")
final_result = self._post_process_hybrid(hybrid_result)

logger.info(f"Hybrid correlation complete:
{final_result['cluster'].nunique()} clusters")
return final_result

def _run_union_find(
    self,
    data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str]
) -> pd.DataFrame:
    """Run Union-Find correlation with adaptive threshold."""
    uf_result = union_find_correlation(
        data,
        usernames,
        addresses,
        use_adaptive_threshold=True
    )

    # Add method标识
    uf_result['method'] = 'union_find'
    uf_result['uf_confidence'] = self._calculate_uf_confidence(uf_result,
usernames, addresses)

    return uf_result

def _run_hgnn(
    self,
    data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str]
) -> pd.DataFrame:
    """Run HGNN correlation."""
    hgnn_result = self.hgnn_engine.correlate(data, usernames, addresses)

    # Rename confidence column for clarity
    hgnn_result['hgnn_confidence'] = hgnn_result['confidence']
    hgnn_result.drop('confidence', axis=1, inplace=True)

```

```

        return hgnn_result

def _combine_results(
    self,
    uf_result: pd.DataFrame,
    hgnn_result: pd.DataFrame
) -> pd.DataFrame:
    """
    Combine Union-Find and HGNN results using weighted ensemble.

    Strategy:
    - For high-confidence HGNN results: trust HGNN
    - For low-confidence HGNN results: blend with Union-Find
    - For conflicting assignments: use weighted voting
    """
    combined = uf_result.copy()
    combined['hgnn_cluster'] = hgnn_result['cluster']
    combined['hgnn_confidence'] = hgnn_result['hgnn_confidence']

    # Initialize hybrid cluster assignments
    combined['hybrid_cluster'] = 0

    # Case 1: High confidence HGNN results - trust HGNN
    high_conf_mask = hgnn_result['hgnn_confidence'] >=
self.confidence_threshold
    combined.loc[high_conf_mask, 'hybrid_cluster'] =
hgnn_result.loc[high_conf_mask, 'cluster']

    # Case 2: Low confidence HGNN results - blend with Union-Find
    low_conf_mask = ~high_conf_mask
    if low_conf_mask.any():
        blended_clusters = self._blend_low_confidence_results(
            combined.loc[low_conf_mask],
            uf_result.loc[low_conf_mask],
            hgnn_result.loc[low_conf_mask]
        )
        combined.loc[low_conf_mask, 'hybrid_cluster'] = blended_clusters

    # Case 3: Handle conflicts between methods
    combined = self._resolve_cluster_conflicts(combined)

    # Renumber clusters sequentially
    combined['cluster'] = self._renumber_clusters(combined['hybrid_cluster'])
    combined.drop(['hybrid_cluster'], axis=1, inplace=True)

    return combined

def _blend_low_confidence_results(
    self,
    combined_data: pd.DataFrame,
    uf_data: pd.DataFrame,
    hgnn_data: pd.DataFrame
) -> List[int]:
    """

```

```

        Blend Union-Find and HGNN results for low-confidence alerts.

        Uses weighted voting based on confidence scores and method weights.
        """
        blended_clusters = []

        for idx in combined_data.index:
            uf_cluster = uf_data.loc[idx, 'cluster']
            hgnn_cluster = hgnn_data.loc[idx, 'cluster']
            hgnn_confidence = hgnn_data.loc[idx, 'hgnn_confidence']
            uf_confidence = uf_data.loc[idx, 'uf_confidence']

            # Calculate weighted scores
            hgnn_score = hgnn_confidence * self.hgnn_weight
            uf_score = uf_confidence * self.union_find_weight

            # Weighted voting
            if hgnn_score > uf_score:
                blended_clusters.append(hgnn_cluster)
            else:
                blended_clusters.append(uf_cluster)

        return blended_clusters

def _resolve_cluster_conflicts(self, combined: pd.DataFrame) -> pd.DataFrame:
    """
    Resolve conflicts between Union-Find and HGNN assignments.

    Strategy: Create consensus clusters based on entity co-occurrence.
    """
    # Find alerts where UF and HGNN disagree
    conflict_mask = combined['cluster'] != combined['hgnn_cluster']

    if not conflict_mask.any():
        return combined

    # For each conflict, examine entity relationships
    conflict_indices = combined[conflict_mask].index

    for idx in conflict_indices:
        uf_cluster = combined.loc[idx, 'cluster']
        hgnn_cluster = combined.loc[idx, 'hgnn_cluster']

        # Check which cluster has more related entities
        uf_entity_score = self._calculate_cluster_entity_support(combined,
                                                                    idx, uf_cluster)
        hgnn_entity_score = self._calculate_cluster_entity_support(combined,
                                                                    idx, hgnn_cluster)

        # Choose cluster with higher entity support
        if hgnn_entity_score > uf_entity_score:
            combined.loc[idx, 'hybrid_cluster'] = hgnn_cluster
        else:
            combined.loc[idx, 'hybrid_cluster'] = uf_cluster

```

```

        return combined

def _calculate_cluster_entity_support(
    self,
    combined: pd.DataFrame,
    alert_idx: int,
    cluster_id: int
) -> float:
    """
    Calculate entity support score for alert-cluster assignment.

    Higher score means more entities in common with cluster members.
    """
    alert_row = combined.loc[alert_idx]
    cluster_members = combined[combined['cluster'] == cluster_id]

    # Extract entities from alert
    alert_entities = self._extract_alert_entities(alert_row)

    # Calculate entity overlap with cluster
    total_support = 0.0
    entity_types = ['SourceAddress', 'DestinationAddress', 'SourceUserName',
'SourceHostName']

    for entity_type in entity_types:
        if entity_type not in alert_row:
            continue

        alert_entity = alert_row[entity_type]
        if pd.isna(alert_entity) or alert_entity in ['UNKNOWN', '0.0.0.0']:
            continue

        # Count occurrences in cluster
        cluster_entities = cluster_members[entity_type].dropna()
        entity_count = (cluster_entities == alert_entity).sum()

        # Weight by entity type importance
        if 'Address' in entity_type:
            weight = 0.4 # IP addresses are important
        elif 'UserName' in entity_type:
            weight = 0.4 # Usernames are important
        else:
            weight = 0.2 # Hostnames are less important

        total_support += entity_count * weight

    return total_support

def _extract_alert_entities(self, alert_row: pd.Series) -> Dict[str, str]:
    """Extract entities from alert row."""
    entities = {}

    entity_columns = [

```

```

        'SourceAddress', 'DestinationAddress', 'DeviceAddress',
        'SourceUserName', 'DestinationUserName',
        'SourceHostName', 'DestinationHostName', 'DeviceHostName'
    ]

    for col in entity_columns:
        if col in alert_row:
            entities[col] = alert_row[col]

    return entities

def _calculate_uf_confidence(
    self,
    uf_result: pd.DataFrame,
    usernames: List[str],
    addresses: List[str]
) -> np.ndarray:
    """
    Calculate confidence scores for Union-Find clusters.

    Based on cluster cohesion and entity support.
    """
    confidences = np.zeros(len(uf_result))

    for cluster_id in uf_result['cluster'].unique():
        cluster_mask = uf_result['cluster'] == cluster_id
        cluster_data = uf_result[cluster_mask]

        # Calculate cluster cohesion
        cohesion_score = self._calculate_cluster_cohesion(cluster_data,
            usernames, addresses)

        # Assign confidence to all alerts in cluster
        confidences[cluster_mask] = cohesion_score

    return confidences

def _calculate_cluster_cohesion(
    self,
    cluster_data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str]
) -> float:
    """
    Calculate cohesion score for a Union-Find cluster.

    Higher score indicates more cohesive cluster.
    """
    if len(cluster_data) <= 1:
        return 0.5 # Neutral confidence for singletons

    # Entity diversity metrics
    unique_entities = set()

```

```

# Count unique entities
for addr_col in addresses:
    if addr_col in cluster_data.columns:
        unique_entities.update(cluster_data[addr_col].dropna().unique())

for user_col in usernames:
    if user_col in cluster_data.columns:
        unique_entities.update(cluster_data[user_col].dropna().unique())

# Cohesion based on entity overlap
total_possible_entities = len(addresses) + len(usernames)
entity_ratio = len(unique_entities) / total_possible_entities if
total_possible_entities > 0 else 0

# Normalize to [0, 1]
cohesion = max(0.1, min(1.0, 1.0 - entity_ratio))

return cohesion

def _post_process_hybrid(self, hybrid_result: pd.DataFrame) -> pd.DataFrame:
    """Post-process hybrid correlation results."""
    # Add hybrid method标识
    hybrid_result['method'] = 'hybrid'

    # Calculate final confidence scores
    hybrid_result['confidence'] =
self._calculate_hybrid_confidence(hybrid_result)

    # Remove intermediate columns
    columns_to_drop = ['hgnn_cluster', 'hgnn_confidence', 'uf_confidence']
    for col in columns_to_drop:
        if col in hybrid_result.columns:
            hybrid_result.drop(col, axis=1, inplace=True)

    return hybrid_result

def _calculate_hybrid_confidence(self, hybrid_result: pd.DataFrame) ->
np.ndarray:
    """Calculate final confidence scores for hybrid results."""
    confidences = np.zeros(len(hybrid_result))

    for cluster_id in hybrid_result['cluster'].unique():
        cluster_mask = hybrid_result['cluster'] == cluster_id
        cluster_size = cluster_mask.sum()

        # Confidence based on cluster size and method consistency
        size_confidence = min(cluster_size / 10.0, 1.0) # Normalize by
expected cluster size

        # Check method consistency within cluster
        if 'method' in hybrid_result.columns:
            methods = hybrid_result.loc[cluster_mask, 'method'].value_counts()
            consistency_score = methods.max() / cluster_size
        else:

```



```

        consistency_score = 1.0

        # Combined confidence
        final_confidence = (size_confidence + consistency_score) / 2.0
        confidences[cluster_mask] = final_confidence

    return confidences

def _renumber_clusters(self, cluster_assignments: pd.Series) -> pd.Series:
    """Renumber clusters sequentially starting from 1."""
    unique_clusters = sorted(cluster_assignments.unique())
    cluster_mapping = {old: new + 1 for new, old in
        enumerate(unique_clusters)}

    return cluster_assignments.map(cluster_mapping)

def enhanced_correlation_hgnn(
    data: pd.DataFrame,
    usernames: List[str],
    addresses: List[str],
    model_path: Optional[str] = None,
    device: str = 'cuda' if torch.cuda.is_available() else 'cpu',
    confidence_threshold: float = 0.5,
    fallback_to_union_find: bool = True
) -> pd.DataFrame:
    """
    Drop-in replacement for correlation_indexer.enhanced_correlation()
    Uses HGNN instead of Union-Find for clustering.

    Args:
        data: DataFrame with security events
        usernames: List of username column names
        addresses: List of IP address column names
        model_path: Path to trained HGNN model
        device: Device to run inference on
        confidence_threshold: Confidence threshold for HGNN
        fallback_to_union_find: Whether to use Union-Find as fallback

    Returns:
        DataFrame with HGNN cluster assignments
    """
    if model_path is None:
        raise ValueError("model_path is required for HGNN correlation")

    if not Path(model_path).exists():
        raise FileNotFoundError(f"HGNN model not found at {model_path}")

    try:
        # Initialize HGNN engine
        hgnn_engine = HGNNCorrelationEngine(
            model_path=model_path,
            device=device,
            confidence_threshold=confidence_threshold
        )

```

```
# Run HGNN correlation
result_df = hgnn_engine.correlate(data, usernames, addresses)

logger.info(f"HGNN correlation complete: {result_df['cluster'].nunique()}
clusters")

return result_df

except Exception as e:
    logger.error(f"HGNN correlation failed: {e}")

if fallback_to_union_find:
    logger.info("Falling back to Union-Find correlation")
    return union_find_correlation(data, usernames, addresses)
else:
    raise
```

Performance Comparison

Method	Speed	Accuracy	Training	GPU	Best For
Union-Find	~100ms/1K alerts	Rule-based	None	No	Small datasets (<100 events), real-time processing
HGNN	~2s/1K alerts	86.45%	Required	Optional	Large datasets (>1000 events)
Hybrid	~500ms/1K alerts	~85%	Required	Optional	Medium datasets (100-1000 events)

Performance Characteristics

Union-Find:

- **Time Complexity:** $O(n \alpha(n))$ - nearly linear
- **Space Complexity:** $O(n)$
- **Scalability:** Excellent for real-time processing
- **Accuracy:** Deterministic based on threshold rules
- **Memory:** Low, stores only parent pointers

HGNN:

- **Time Complexity:** $O(n + e)$ per forward pass
- **Space Complexity:** $O(n + e)$ for graph storage
- **Scalability:** GPU acceleration for large graphs
- **Accuracy:** Learned representations, 86.45% campaign prediction
- **Memory:** Higher, stores node features and edge indices

Hybrid:

- **Time Complexity:** Union-Find + HGNN time
 - **Space Complexity:** Combined requirements
 - **Scalability:** Balanced approach
 - **Accuracy:** Weighted ensemble, ~85% effective accuracy
 - **Memory:** Moderate, uses both approaches selectively
-

Integration Examples

Basic Usage

```
from core.correlation_pipeline import CorrelationPipeline

# Auto method selection based on data size
pipeline = CorrelationPipeline(method='auto')
result = pipeline.correlate(alerts_df, ['SourceUserName'], ['SourceAddress'])

print(f"Method used: {result.method_used}")
print(f"Clusters found: {result.num_clusters}")
print(f"Runtime: {result.runtime_seconds:.2f}s")
```

Method-Specific Usage

```
# Union-Find for real-time processing
uf_pipeline = CorrelationPipeline(method='union_find')
uf_result = uf_pipeline.correlate(alerts_df, ['SourceUserName'],
                                  ['SourceAddress'])

# HGNN for high accuracy
hgnn_pipeline = CorrelationPipeline(
    method='hgnn',
    model_path='hgnn_checkpoints/multidomain_v2/best_supervised.pt'
)
hgnn_result = hgnn_pipeline.correlate(alerts_df, ['SourceUserName'],
                                       ['SourceAddress'])

# Hybrid for balanced approach
hybrid_pipeline = CorrelationPipeline(
    method='hybrid',
    model_path='hgnn_checkpoints/multidomain_v2/best_supervised.pt',
    hgnn_weight=0.7,
    union_find_weight=0.3
)
hybrid_result = hybrid_pipeline.correlate(alerts_df, ['SourceUserName'],
                                           ['SourceAddress'])
```

Advanced Configuration

```
# Custom confidence thresholds
pipeline = CorrelationPipeline(
    method='auto',
    confidence_threshold=0.8, # Higher threshold for more conservative clustering
    hgnn_weight=0.8,         # Favor HGNN more strongly
    union_find_weight=0.2
)

# With preprocessing
from core.preprocessing import AlertPreprocessor

preprocessor = AlertPreprocessor()
processed_data = preprocessor.preprocess_data(alerts_df)

result = pipeline.correlate(processed_data, ['SourceUserName'], ['SourceAddress'])
```

Output Generation

```
from core.output import OutputGenerator

# Generate comprehensive output
output_generator = OutputGenerator(output_dir="analysis_results")
output_files = output_generator.generate_comprehensive_output(
    result.data,
    result,
    output_prefix="security_analysis_2024"
)

print("Generated files:")
for output_type, file_path in output_files.items():
    print(f"  {output_type}: {file_path}")
```

Batch Processing

```
# Process multiple datasets
datasets = {
    'unsw': unsw_df,
    'beth': beth_df,
    'optc': optc_df
}

results = {}
for dataset_name, dataset_df in datasets.items():
    print(f"Processing {dataset_name}...")

    pipeline = CorrelationPipeline(method='auto')
    result = pipeline.correlate(dataset_df, ['SourceUserName'], ['SourceAddress'])
```

```
    results[dataset_name] = result
    print(f"    {result.num_clusters} clusters found in
    {result.runtime_seconds:.2f}s")

# Compare results across datasets
for dataset_name, result in results.items():
    print(f"{dataset_name}: {result.method_used} - {result.num_clusters}
    clusters")
```

This comprehensive architecture documentation provides detailed code examples for each component of the MITRE-CORE system, enabling system architects and developers to understand and implement the full pipeline effectively.